CSCI-6730 OPERATING SYSTEMS

Project #1 – Performance Comparison of Serial, Multi-process, and Multi-threaded Applications in Matrix Multiplication

HRUTHI MUGGALLA

ID: 811495373

INTRODUCTION

In this project, we design and implement a multi process and multi thread matrix multiplication program by converting a given single-process matrix multiplication in C language. It aims to compare execution times of matrix multiplication using three approaches: serial processing, multi-processing, and multi-threading by analysing how the application performance under different matrix dimensions. We also perform crash recovery in case of crashes to bring back the system to a safe state where the system can operate normally.

IMPLEMENTATION OF MULTI PROCESS APPLICATION

This design performs parallel matrix multiplication using several processes, which it achieves through the system call fork(). Here inter-process communication is done using shared memory. The design also performs crash recovery by the creation of new processes until the process completes its execution. The design is scalable, and performance is optimized for larger dimensions of matrices as it distributes the workload amongst several processes.

Core functions:

1. mat_multi_proc:
    - Function performs matrix multiplication with the help of a number of child processes. It initializes shared memory for storing results.
    - It monitors child processes and restarts them if any of the child processes happens to crash during execution. It keeps restarting the process every time till it completes its execution.

- After completion, results are copied to the final matrix and shared memory resources are cleared.

  Shared memory initialization:

```
shared_mem_key = shmget(IPC_PRIVATE, m * p * sizeof(int), IPC_CREAT | 0666);
if (shared_mem_key < 0)
{
    perror("shmget");
    exit(1);
}

shared_output = (int *)shmat(shared_mem_key, NULL, 0);
if (shared_output == (int *)-1)
{
    perror("Failed to attach shared memory");
    exit(1);
}
```

  Process creation:

```
for(proc_idx = 0; proc_idx < numProc; proc_idx++)
{
    child_pids[proc_idx] = fork();
    if(child_pids[proc_idx] == 0) {
        child_process_core(proc_idx, numProc, shared_output);
        exit(0);
    } else if(child_pids[proc_idx] < 0) {
            printf("Fork failed\n");
            exit(0);
    }
}
```

2. Child_process_core
   - Entry point for each child process - decides which rows to compute based on the amount of processes in total.
   - Simulates crash to test the crash recovery mechanism.
   - Calls compute_multiply_matrices to compute its assigned rows and stores its results in shared memory.

```c
void child_process_core(int proc_id,  int numProc, int *shared_output)
{
        int rows_per_proc = m / numProc;
        int start_idx = proc_id * rows_per_proc;
        int end_idx = (proc_id + 1) * rows_per_proc - 1;
        if (proc_id == numProc - 1)
        {
            end_idx = m - 1;
        }
        printf("The child process (pid:%d) created to calculate job #(%d/%d).\n", getpid(),
        proc_id, m);
        simulate_process_crash(proc_id);

        compute_multiply_matrices(start_idx, end_idx, shared_output);
}
```

3. Compute_multiply_matrices
   - Multiplies two matrices for a given range of rows, placing results in shared array.
   - It calculates the dot product of each row taken from matrix A with all columns of the transposed matrix B, B_tran.
   - This function is called by the child processes in order to handle their portion of the matrix multiplication.

```c
void compute_multiply_matrices(int start_idx, int end_idx, int *shared_output)
{
    int row_idx, col_idx, k_idx;
    for (row_idx = start_idx; row_idx <= end_idx; row_idx++)
    {
        for (col_idx = 0; col_idx < p; col_idx++)
        {                                   int col_idx
            shared_output[row_idx * p + col_idx] = 0;
            for (k_idx = 0; k_idx < n; k_idx++)
            {
                shared_output[row_idx * p + col_idx] += A[row_idx][k_idx] * B_tran[col_idx][k_idx];
            }
        }
    }
}
```

IMPLEMENTATION OF MULTI THREAD APPLICATION

This design also implements a parallelized matrix multiplication approach using multiple threads, using shared memory for efficient data access and inter-thread communication. Besides, it can recover from a crash if the threads fail during execution, re-creating the threads-hence adding robustness to the computations. This design efficiently distributes the workload among threads in order to

optimize performance for large matrix operations while ensuring all tasks are accomplished successfully. This design also introduces a mutex lock that is used to guard shared access to the done array, thereby safely updating the thread completion status without any race conditions.

Core functions:

1. Child_thread_core
   - Takes pointer as an argument to a Thread structure that contains the index of the thread, the total number of threads, a flag about the task completion status, and a mutex

```
Thread *data = (Thread *)arg;
long index = data->idx;
int numThread = data->numThread;
int *done = data->done;
pthread_mutex_t *mutex = data->mutex;

printf("The child thread[%ld] created to calculate job #%ld.\n", index, index);
```

   - It implements crash recovery. It updates the done array if thread did not complete the task and recreates the thread till it does not crash and completes its execution.

```
if (rand() % 100 < crashRate) {
    printf("Thread[%ld] crashed.\n", index);
    pthread_mutex_lock(&mutex[index]);
    done[index] = 0;
    pthread_mutex_unlock(&mutex[index]);
    pthread_exit(NULL);
}
```

- Once the thread finishes the multiplication of the matrix, it locks the mutex, changes the status in the done array to mark that the task has been finished, and then unlocks the mutex before its termination.

```
printf("Thread[%ld] completed work\n", index);
pthread_mutex_lock(&mutex[index]);
done[index] = 1;
pthread_mutex_unlock(&mutex[index]);
return NULL;
```

2. Mat_mult_multi_thread:

- It dynamically creates memory for the thread objects: threads and threadArray, an array to track each thread completion status named done and an array called mutex.

```
pthread_t *threads = malloc(numThread * sizeof(pthread_t));
Thread *threadArray = malloc(numThread * sizeof(Thread));
int *done = malloc(numThread * sizeof(int));
pthread_mutex_t *mutex = malloc(numThread * sizeof(pthread_mutex_t));
int retries[numThread];
int i;
```

- This function, for each thread, populates the Thread structure and creates a thread through calling pthread_create() to execute the child_thread_core() function

```
for (i = 0; i < numThread; i++) {
    threadArray[i].idx = i;
    threadArray[i].numThread = numThread;
    threadArray[i].done = done;
    threadArray[i].mutex = mutex;
    pthread_create(&threads[i], NULL, child_thread_core, &threadArray[i]);
}
```

- The parent thread enters a loop in order to monitor the completion of all child threads. If a thread has crashed i.e done[i] == 0 , it joins the

failed thread and then recreates it in order to retry the task, until all threads have completed their task successfully.

```
int allDone = 0;
while (!allDone) {
    allD
                int pthread_mutex_lock(pthread_mutex_t *__mutex)
                Lock a mutex.
    for
        pthread_mutex_lock(&mutex[i]);
        if (done[i] == 0 ) {
            pthread_mutex_unlock(&mutex[i]);
            pthread_join(threads[i], NULL);
            pthread_mutex_lock(&mutex[i]);
            if (done[i] == 0) {
            printf("Recreating thread[%d] to complete the task.\n", i);
            pthread_create(&threads[i], NULL, child_thread_core, &threadArray[i]);
            allDone = 0;
        }

        }
        pthread_mutex_unlock(&mutex[i]);
    }
}
```

- Once all tasks are done, it joins all the remaining threads, in order for all the resources to be handled appropriately. Then, it destroys the mutexes and frees the dynamically allocated memory - threads, mutexes, and the array done.

```
for (i = 0; i < numThread; i++) {
    if(done[i] ==0)
    {
        pthread_join(threads[i], NULL);
    }
}

printf("All threads completed successfully.\n");

 for (i = 0; i < numThread; i++) {
    pthread_mutex_destroy(&mutex[i]);
}
```

PERFORMANCE EVALUATION

The performance evaluation strategy includes matrix multiplication across a range of dimensions of matrices which correspond with the number of threads applied using serial, multi-process, and multi-threaded applications. Various matrix sizes are used so as to assess the scalability of each approach with increased workload and measure execution times for each method.

Observations and Results

	Dimensions of Matrix	Serial	Multi Process	Multi Thread
Test 1	A – 10 x 5 B – 5 x 5	0.000001	0.001695	0.000381
Test 2	A – 5 x 10000 B – 10000 x 10000	1.158336	0.486466	0.229857
Test 3	A – 1 x 50 B – 50 x 5000	0.000561	0.002011	0.000689
Test 4	A – 30 x 50 B – 50 x 1000	0.003358	0.005125	0.002472
Test 5	A – 5 x 10 B – 10 x 500	0.000063	0.000690	0.000225
Test 6	A – 15 x 10 B – 10 x 500	0.001891	0.003584	0.001196
Test 7	A – 20 x 100 B – 100 x 50	0.000326	0.003890	0.000747
Test 8	A – 1 x 100 B – 100 x 10000	0.002330	0.005661	0.002418
Test 9	A – 1 x 1 B – 1 x 1	0.000000	0.000810	0.000114

Analysis of Results:

a. Performance Trends:

- As the size of the matrices increases-significantly (test2.txt and test8.txt), execution time through the serial approach is drastically bigger compared

to multi-process and multi-threaded ones. This reflects the advantage of parallelization, when handling big datasets.

- Multi-threaded implementation ran faster compared to multi-processes in almost all instances. Hence, threading provided better resource utilization with less overhead compared to the creation of separate processes for the tasks.

b. Unexpected Results:

- Minor performance differences in tests with the smaller matrices (test1.txt and test5.txt.). This observation says that parallelism is more beneficial when we are dealing with larger datasets.

c. Insights:

- It points out that the size of the matrix and the computation load determine the choice between multi-processing and multi-threading. For larger datasets, performance makes multi-threading preferred; for smaller matrices, the overhead may not justify parallel execution.

CRASH RECOVERY STRATEGY

The design runs several child processes and checks for their execution status. It enters a loop to wait for each of the processes to complete. If any of the processes crash or exit with an error, it logs a message and attempts to restart that process. It keeps restarting the process till the process executes successfully.

MULTI PROCESS

```
for(proc_idx = 0; proc_idx < numProc; proc_idx++)
{
        if (waitpid(child_pids[proc_idx], &proc_status, 0) != -1)
    {
        if (!WIFEXITED(proc_status) || WEXITSTATUS(proc_status) != 0)
        {
            printf("Process %d crashed. Retrying...\n", child_pids[proc_idx]);
            if ((child_pids[proc_idx] = fork()) == 0)
            {
                child_process_core(proc_idx, numProc, shared_output);
                exit(0);
            }
            proc_idx--;  // Ensure retried process is handled properly
        }
    }
}
```

```
The child process (pid:23543) created to calculate job #(0/4).
The child process (pid:23544) created to calculate job #(1/4).
*** Process[1] (pid 23544) crashed.
The child process (pid:23545) created to calculate job #(2/4).
The child process (pid:23546) created to calculate job #(3/4).
*** Process[3] (pid 23546) crashed.
Process 23544 crashed. Retrying...
The child process (pid:23550) created to calculate job #(1/4).
Process 23546 crashed. Retrying...
The child process (pid:23594) created to calculate job #(3/4).
*** Process[3] (pid 23594) crashed.
Process 23594 crashed. Retrying...
The child process (pid:23596) created to calculate job #(3/4).
==================================================
```

MULTI THREAD

```c
int allDone = 0;
while (!allDone) {
    allDone = 1;

    for (i = 0; i < numThread; i++) {
        pthread_mutex_lock(&mutex[i]);
        if (done[i] == 0 ) {
            pthread_mutex_unlock(&mutex[i]);
            pthread_join(threads[i], NULL);
            pthread_mutex_lock(&mutex[i]);
            if (done[i] == 0) {
            printf("Recreating thread[%d] to complete the task.\n", i);
            pthread_create(&threads[i], NULL, child_thread_core, &threadArray[i]);
            allDone = 0;
        }

        }
        pthread_mutex_unlock(&mutex[i]);
    }
}
```

```
The child thread[0] created to calculate job #0.
Thread[0] completed work
The child thread[1] created to calculate job #1.
Thread[1] completed work
The child thread[2] created to calculate job #2.
Thread[2] completed work
The child thread[3] created to calculate job #3.
Thread[3] crashed.
Recreating thread[3] to complete the task.
The child thread[3] created to calculate job #3.
Thread[3] completed work
All threads completed successfully.
==================================================
```